

Designspecifikation Snake44

TSEA83 – Grupp 44

Ferdinand Pergel – ferpe211

Mhd Khaled Helwany- mhdhe919

Adam Ljung - adalj586

Melker Karlsson – melka028

Version 0.1

Implementation

I projektet kommer det att finnas två olika minnen, ett programminne som ansvarar för flödet av programmet. Ett minne som är delat för att spara data och information om grafiken(tiles). Ormen styrs genom knapptryck från tangentbordet, därav kommer det implementeras en tangentbordskodare som hanterar knapptryck. I mån av tid ska en ljud enhet implementeras.

Att hålla koll på ormens position med avseende på dynamisk storlek kan vara lite klurig. Preliminära lösningen är att alla celler är länkade till den framför, vid nytt tick så tar cellen nästa cells position, huvudet kan vara en “speciell tile” eftersom dess nästvarande position beror på spelarens inmatning.

Processorn

Vi kommer att bygga en mikroprogrammerad processor för vår Snake-implementation. Processorn använder delat minne för effektiv datahantering. Den naturliga ordbredden för processorn är 16 bitar, vilket ger tillräcklig kapacitet för våra beräkningsbehov. Vi ska kunna hantera 10 bitar som motsvarar ett pixelvärde för x- eller y-koordinat och pixelvärden varierar mellan 0 och 640. Det enda minnet som skiljer sig från 16 bitar är mikrominnet som är 32 bitar bred. Detta för att kunna hantera de operationer vi vill kunna göra.

Processorn stödjer tre olika adresseringsmoder: Immediate, Direct och Indexerad adressering. Detta ger oss flexibilitet för olika typer av dataåtkomst i vårt program. Processorn kommer att starta med ett förladdat program från början, vilket förenklar implementationen.

Vad gäller arbetsuppgifter kommer CPU:n att ansvara för kollisionshantering för ormarna, pseudorandom generering av mat på spelplanen, samt övrig spellogik. Processorns prestanda kommer vara tillräckligt för dessa relativt enkla beräkningar som spelet kräver.

Instruktionsset

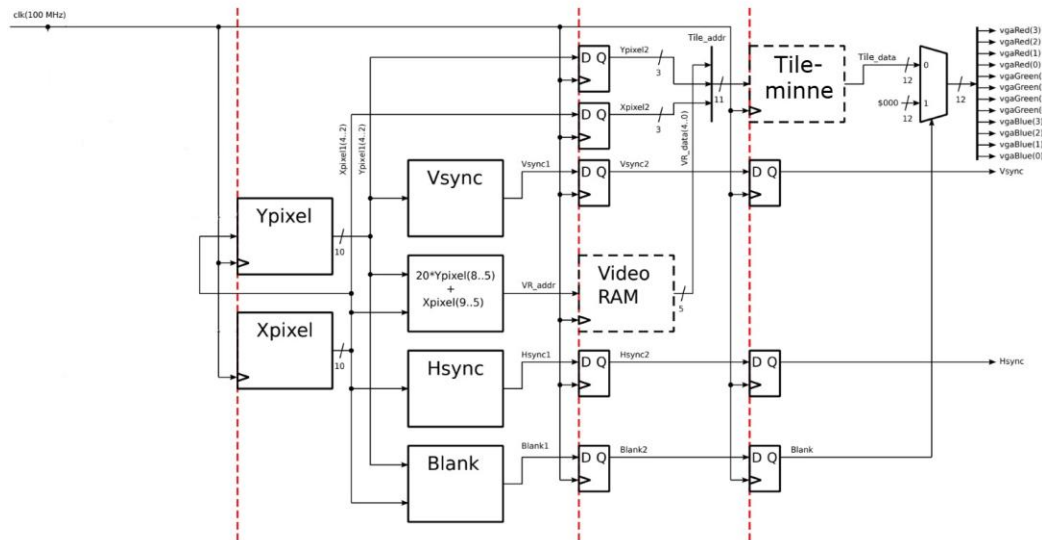
Vi tänker oss att följande instruktioner kommer behövas:

Mnemonic	Operander	Flaggor	Funktion
LDI	GR _x , VALUE	-	GR _x := VALUE
LD	GR _x , M, ADR	-	GR _x := DM(A)
STORE	GR _x , M, ADR	-	PM(A) := GR _x
ADD	GR _x , M, ADR	Z, N, C, V	GR _x := GR _x + DM(A)
SUB	GR _x , M, ADR	Z, N, C, V	GR _x := GR _x - DM(A)
OR	GR _x , M, ADR	Z, N	GR _x := GR _x OR DM(A)
LSR	GR _x , M, ADR	Z, N, C	GR _x := GR _x >> 1 eller DM(A) := DM(A) >> 1
CMP	GR _x , M, ADR	Z, N, C, V	GR _x - PM(A) (sätt flaggor)
BRA	M, ADR	-	PC := A
BEQ	M, ADR	-	Om Z=1: PC := A
MOV	GR _x , GR _y	Z, N	GR _x := GR _y
BPL	M, ADR	-	Om N=0: PC := A
KEY	GR _x , M, ADR	-	GR _x := KeyboardRegister(ADR)

Grafik

Vår grafikstrategi använder en enkel tilebaserad VGA-controller för Snake-spelet. Vi implementerar ett kompakt tileminne med endast de nödvändiga mönstren (ormhuvuden för båda spelarna, kroppssegment i olika färger, och mat) samt ett tilemap-minne som representerar spelplanen. CPU:n uppdaterar endast tilemap-minnet när ormarnas positioner ändras eller mat placeras, medan VGA-controllern kontinuerligt läser detta minne för att generera videosignalen. minimala resurser. Vi har tänkt använda oss av sju unika färger för exempelvis spelarkroppar, bakgrund, mat osv. Vi har planerat att använda oss av en upplösning på 640 x 480 pixlar.

Nedan finns blockschema över vår strategi att rita grafik.



I/O-enheter

Tangentbord: En keyboard encoder ska hantera avläsning av knapptryckningar så att spelaren kan styra ormen. Tangentbordet kommer att kopplas in via USB.

VGA monitor: En VGA motor ska hantera ritning av pixlarna genom att skapa synkroniseringssignaler och hålla koll på pixelns koordinat som ska ritas vid en viss klockcykel.

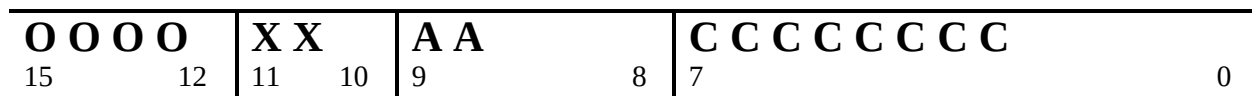
Piezo (ljudenhet): Datan för olika ljud kommer att lagras i dataminnet och skickas till Piezo vid lämpliga ställen under körningens gång.

Minnesanvändning

Vår CPU-design inkluderar flera typer av minnen som alla ryms inom Basys3-kortets 1800 Kbit block RAM. Vi använder fyra 16-bitars generella register (totalt 64 bitar), ett programminne (PM) som CPU:n behöver läsa från, samt ett 16-bitars dataminne som kräver både läs- och skrivåtkomst. Adressregistren PC och ADR är båda 16 bitar breda, vilket ger en teoretisk adressrymd på 65536 ord, men i praktiken räcker det med 1K-4K ord (16-64 Kbit) för både program- och dataminne i den här implementationen. Instruktionsregistret (IR) är 16 bitar brett för att lagra den aktuella instruktionen. Vi har även ett tileminne som har plats för 4096 olika tiles, där varje tile är 16 bitar bred. Av dessa 16 bitar används 10 bitar för pixelvärde och 6 bitar för färg. Med dessa specifikationer uppskattar vi den totala minnesåtgången till mindre än 150 Kbit, vilket utgör mindre än 10% av Basys3-kortets tillgängliga block RAM, vilket ger god marginal för både implementering och eventuella framtida utökningar.

I vår CPU-design implementerar vi ett instruktionsregister med följande bitallokering:

OP-Kod	GRx	Addressmod	ADR/Const
--------	-----	------------	-----------



Avkodaren består av ett rent kombinatoriskt nätverk som analyserar inkommande instruktioner och producerar styrsignaler till CPU:ns olika komponenter. Baserat på den aktuella operationskoden genereras specifika aktiverings signaler som får processorns byggblock att utföra den önskade funktionen. Vilka utsignaler som avkodaren behöver generera bestäms direkt av instruktionsuppsättningen som implementeras i processorn.

Programmering

Vårt system kommer att programmeras genom mikroprogrammering av mikrodatorn, där VHDL används för all hårdvaruimplementering inklusive CPU, minnesmoduler och I/O-gränssnitt. För att undvika den mödosamma processen att skriva instruktioner direkt i binärkod kommer vi att utveckla en Python-baserad interpreter som översätter assemblerprogram till binärkod anpassad för vår arkitektur. Programmeringsflödet kommer att bestå av att först skriva program i vårt egna assemblerspråk, som sedan konverteras till binärkod av vår Python-interpreter och laddas in i programminnet. Mikroprogram som definierar hur varje instruktion utförs kommer att implementeras direkt i VHDL och lagras i mikrominnet.

Timing

Vid varje ny iteration av spelplanen så uppräknar vi en viss mängd ticks i en for loop i assembler. Antalet ticks som for loopen räknar upp till kommer att bestämmas vid testning. Då for loopen har räknat upp det specificerade antalet ticks så uppdateras grafiken för spelplanen.

Boot-loader

Vi planerar att implementera en enkel boot-loader vars enda uppgift är att ta emot binära filer via UART, placera dem i rätt minnesområde och sedan starta exekveringen. Boot-loadern ska vara programmerat i datorn från början, vilket möjliggör snabb uppdatering av programkod utan att behöva omprogrammera hela FPGA:n. Om tidsbegränsningar uppstår eller vi bedömer att funktionaliteten inte är kritisk för projektet, kan vi prioritera bort denna feature till förmån för andra aspekter av CPU-designen.

Milstolpe

Efter att halva av projektets tid har gått ska vi kunna visa en grafisk bild av spelplanen på en VGA-skärm och ha konstruerat en kompilator som tar assemblerkod och översätter den genom Python kod som beskrivet i programmerings delen ovan.

Blockschema

Nedan finns ett blockschema som beskriver vår konstruktion

